Midterm Project — Reflection Journal

Used Car Price Prediction: Dataset Preparation & Team Contributions

ITAI 1371 | Spring 2026

GROUP 5 – HUY NGUYEN, ABRAHAM BARRETO, SANDHYA CHAMAKURI

Project Overview

For this midterm project, our team worked with the Used Car Listings dataset sourced from Kaggle, containing 9,582 records of second-hand vehicle listings from India. Our objective was to clean and prepare the data for a supervised machine learning task: predicting the asking price (AskPrice) of a used car based on its attributes including brand, mileage, age, transmission, fuel type, and ownership history.

The project required significant data preparation effort before any modeling could begin. The raw dataset contained string-encoded numeric fields, duplicate rows, missing values, and high-cardinality categorical columns. Each team member took ownership of a distinct phase of the pipeline, allowing us to work in parallel and combine our results into a clean, ML-ready dataset of 5,973 rows and 26 features.

Team Contribution Summary

Team Member	Primary Responsibility	Key Deliverables
Huy Nguyen	Data Exploration & Cleaning	EDA visualizations, string parsing, null imputation, duplicate removal
Abraham Barreto	Feature Engineering & Encoding	Column filtering, brand analysis, one-hot encoding, train/test split
Sandhya Chamakuri	Scaling & Documentation	StandardScaler pipeline, Jupyter notebook, proposal documents

Huy Nguyen

Data Exploration & Cleaning Lead

Overview of Contribution

My primary responsibility was conducting the initial exploratory data analysis (EDA) and leading the data cleaning phase. I was the first to dig into the raw dataset and identify the issues that would need to be resolved before the data could be used for modeling. This foundational work set the direction for the rest of the team.

What I Accomplished

- **Raw Data Inspection:** I loaded the original CSV and performed a full column-by-column audit. I identified that kmDriven was stored as a string with commas and a ' km' suffix (e.g., '98,000 km'), and that AskPrice contained the Indian Rupee symbol with comma-formatted numbers (e.g., 'Rs 1,95,000'). Neither column could be used numerically without parsing.
- **String Parsing:** I wrote the parsing logic for both kmDriven and AskPrice — stripping the ' km' suffix, removing commas and the Rs symbol, and casting both columns to float. This was a critical step because these two fields are the most important numeric features in the dataset.
- **Missing Value Analysis:** I discovered that 47 rows (roughly 0.5% of the dataset) had null values in the kmDriven column after parsing. I proposed and implemented median imputation as the fix, ensuring the scaler was fitted only on training data to prevent data leakage.
- **Duplicate Removal:** I identified 724 exact duplicate rows in the raw dataset. I removed them using pandas drop_duplicates(), reducing noise and preventing the model from memorizing repeated entries.

- **EDA Visualizations:** I produced the exploratory charts included in the Jupyter notebook: the car age histogram, top-12 brand bar chart, and the fuel type and transmission pie charts. These gave the team a clear picture of the data distributions before cleaning.

Reflection

This phase taught me the importance of never assuming data is clean. The kmDriven and AskPrice columns looked like numbers at a glance but were actually strings, which would have silently broken any downstream model. Running a thorough audit before writing any modeling code saved the team a lot of debugging time later. I also learned that median imputation is the safer choice over mean when a column has potential outliers, since the kmDriven distribution had some vehicles listed with over 900,000 km driven.

Abraham Barreto

Feature Engineering & Encoding Lead

Overview of Contribution

My role centered on transforming the cleaned raw features into a format that machine learning algorithms can actually use. After Huy handled parsing and deduplication, I took over to make the structural decisions about which columns to keep, how to handle high-cardinality categories, and how to encode categorical variables as numeric inputs.

What I Accomplished

- **Train/Test Split:** I implemented the 70/30 train/test split using sklearn's train_test_split with random_state=42 for reproducibility. Critically, I ensured this split happened before any cleaning, encoding, or scaling was applied, so the test set remains a true holdout that has never influenced any pipeline decisions.
- **Column Dropping & Rationale:** I analyzed each column and identified which ones to drop. I removed Year because it is perfectly collinear with Age (keeping both would cause multicollinearity). I removed AdditionInfo and PostedDate because they are either free-text noise or not a property of the car itself. I dropped model because it had hundreds of unique values, making one-hot encoding impractical without extensive preprocessing.
- **Brand Frequency Analysis:** I analyzed all 39 unique brands in the dataset and found that many had fewer than 10 listings each. Rare brands like Aston Martin, Rolls-Royce, and Maserati would produce one-hot columns with almost all zeros, adding noise without contributing signal. I decided to retain only the top 16 brands by listing frequency and filter out rows with rare brands.
- **One-Hot Encoding:** I applied pd.get_dummies() to Brand, Transmission, Owner, and FuelType, specifying dtype=int to ensure clean 0/1 integer output rather than boolean. This produced 23 binary columns that fully represent all categorical information in a numeric form compatible with all our planned models.
- **Pipeline Documentation:** I wrote the detailed comments in the notebook for Steps 4d through 4g, explaining the reasoning behind each decision so that the professor and any future reader can follow the logic.

Reflection

The most challenging decision I had to make was how many brands to keep. Keeping all 39 would have added 39 sparse binary columns, many of which would rarely be 1. I settled on the top 16 after reviewing the frequency distribution and confirming that these brands account for the vast majority of listings. The train/test split timing was also an important lesson: I originally considered splitting after cleaning, but I realized that would allow information from the test set to influence imputation medians and scaler parameters, which is a form of data leakage. Splitting first is the correct approach.

Sandhya Chamakuri

Scaling, Notebook & Documentation Lead

Overview of Contribution

My responsibilities spanned the final stage of the data pipeline and all project documentation. I implemented feature scaling to ensure numeric columns are on comparable scales, assembled the full cleaning pipeline into a polished Jupyter notebook with before-and-after comparisons, and produced both the PDF and Word proposal documents submitted with this project.

What I Accomplished

- **StandardScaler Implementation:** I applied Z-score standardization (mean=0, std=1) to the three continuous numeric columns: Age, kmDriven, and AskPrice. I used sklearn's StandardScaler, fitted exclusively on the training data, and transformed both train and test sets with the same scaler. This ensures no information from the test set influences the scaling parameters.
- **Before vs After Visualization:** I created the 2x3 histogram grid in the notebook that shows each numeric column before and after standardization side by side. This visual comparison makes it easy to see how scaling compresses wide distributions like AskPrice (ranging from Rs 15,000 to Rs 42,500,000) into a standard range.
- **Jupyter Notebook Assembly:** I organized the work from all three team members into a single, well-structured Jupyter notebook with 46 cells (29 code, 17 markdown). Each section includes a BEFORE state, the fix code, and an AFTER state so the reader can clearly see the effect of each transformation. I also wrote the ML Problem Discussion section covering our feature table, model plan, and evaluation metrics.
- **Proposal PDF & Word Document:** I produced the one-page project proposal in both PDF (using Python reportlab) and Word (.docx) formats. Both documents contain the dataset feature table, the 8 data preparation tasks with justifications, and the modelling plan comparing Linear Regression, Random Forest, and XGBoost.
- **Final Dataset Export:** I verified the final cleaned training dataset has zero nulls and zero duplicates, then exported it as final_cleaned_dataset.csv. The output contains 5,973 rows and 26 columns ready for model training.

Reflection

Putting together the notebook was more work than I expected, but it forced me to review every step carefully and make sure the logic was airtight end-to-end. Writing clear before/after comparisons for each cleaning step helped me catch one issue: the scaler was initially being applied before the brand filter, which meant some scaled rows were later dropped. I fixed the order so filtering always happens before scaling. This project reinforced that the order of operations in a data pipeline matters enormously, and that documentation is not an afterthought — it is part of the work.

Team Reflection & Conclusion

Working on this project as a three-person team allowed us to divide the pipeline into natural phases and make progress in parallel. Huy handled raw data exploration and the numeric cleaning tasks, Abraham managed the structural decisions around features and encoding, and Sandhya brought everything together through scaling and documentation. Each contribution was essential: a failure at any stage would have compromised the entire downstream pipeline.

The most valuable overall lesson was the importance of respecting the train/test boundary at every step. Imputation medians, scaler parameters, and brand frequency counts were all computed on training data only and then applied to the test set. This discipline ensures our model evaluation will reflect true out-of-sample performance rather than an optimistically inflated in-sample estimate.

Our cleaned dataset of 5,973 rows and 26 features is ready for the modeling phase. We plan to train Linear Regression as a baseline, followed by Random Forest and XGBoost, evaluating each on MAE, RMSE, and R2 using the held-out 30% test set.